

Section C Tasks

Lab Task 1: Movie Rating Database

Scenario:

You are building a movie streaming service that stores movies by MovieID. Quick search and genre filtering are required for recommendations and management.

Task Requirements:

Each node should contain:

- MovieID (int, unique key)
- Title (string)
- Genre (string)
- Rating (float, 0-10)

Core Functionalities:

- insertMovie(int id, string title, string genre, float rating)
- findMovie(int id)
- deleteMovie(int id)
- displayMoviesInOrder()
- getInorderIDs()

Advanced Operations:

- getTopRatedMovie()
- getMoviesByGenre(string genre) -> vector<Movie>
- getHeight()

Google Test Cases:

Note: Some testcases have vectors in them. You need to replace them with an appropriate list or array

```
TEST(MovieBSTTest, InsertAndFindMovie) {
```

```

MovieBST bst;

bst.insertMovie(101, "Inception", "Sci-Fi", 9.0);

bst.insertMovie(102, "Frozen", "Animation", 8.5);

Movie m = bst.findMovie(102);

EXPECT_EQ(m.Title, "Frozen"); // Expected correct movie

EXPECT_FLOAT_EQ(m.Rating, 8.5);

}

TEST(MovieBSTTest, FindNonExistingMovie) {

    MovieBST bst;

    bst.insertMovie(101, "Inception", "Sci-Fi", 9.0);

    Movie m = bst.findMovie(999);

    EXPECT_EQ(m.Title, ""); // Expected empty when not found

}

TEST(MovieBSTTest, DeleteLeafMovie) {

    MovieBST bst;

    bst.insertMovie(5, "X", "A", 7);

    bst.insertMovie(2, "Y", "B", 8);

    bst.insertMovie(8, "Z", "C", 9);

    bst.deleteMovie(2);

    EXPECT_EQ(bst.findMovie(2).Title, ""); // Expected removal

}

```

```

TEST(MovieBSTTest, DeleteMovieWithTwoChildren) {

    MovieBST bst;

    bst.insertMovie(50, "Root", "A", 6);

    bst.insertMovie(30, "Left", "B", 7);

    bst.insertMovie(70, "Right", "C", 8);

    bst.insertMovie(60, "Mid", "D", 7.5);

    bst.deleteMovie(50);

    EXPECT_EQ(bst.findMovie(50).Title, ""); // Expected replaced root
}

```

```

TEST(MovieBSTTest, InorderSortedIDs) {

    MovieBST bst;

    bst.insertMovie(3, "A", "X", 5);

    bst.insertMovie(1, "B", "Y", 6);

    bst.insertMovie(2, "C", "Z", 7);

    vecto<int> ino = bst.getInorderIDs();

    EXPECT_EQ(ino, (vecto<int>{1,2,3})); // Expected sorted sequence
}

```

```

TEST(MovieBSTTest, GetTopRatedMovie) {

    MovieBST bst;

    bst.insertMovie(11, "MovieA", "Drama", 7.4);

```

```

    bst.insertMovie(12, "MovieB", "Action", 9.1);

    Movie top = bst.getTopRatedMovie();

    EXPECT_EQ(top.Title, "MovieB"); // Expected highest rating
}

TEST(MovieBSTTest, GetMoviesByGenre) {

    MovieBST bst;

    bst.insertMovie(1, "M1", "Drama", 6);

    bst.insertMovie(2, "M2", "Sci-Fi", 8);

    bst.insertMovie(3, "M3", "Sci-Fi", 9);

    vecto<Movie> sci = bst.getMoviesByGenre("Sci-Fi");

    EXPECT_EQ(sci.size(), 2); // Expected two Sci-Fi movies
}

TEST(MovieBSTTest, HeightAfterInsertions) {

    MovieBST bst;

    bst.insertMovie(10, "A", "G", 5);

    bst.insertMovie(5, "B", "G", 6);

    bst.insertMovie(15, "C", "G", 7);

    EXPECT_EQ(bst.getHeight(), 2); // Expected height = 2
}

```

Lab Task 2: Binary Search Tree to Sorted Linked List Conversion

You are part of a software team developing an **in-memory data manager** for a content indexing service.

The service stores items (e.g., documents, files, or user records) in a **Binary Search Tree (BST)** for efficient insertion and search. However, some downstream systems — such as reporting tools, sequential data loaders, or cache optimizers — require **data in a linear, sorted format** (like a linked list or array).

Instead of performing repeated traversals or building temporary arrays, you must implement an **in-place conversion** from the BST structure to a **sorted singly linked list** (or doubly linked list).

The challenge:

- Maintain the sorted order of data.
- Use **O(1)** extra space (other than recursion stack).
- Avoid reconstructing nodes from scratch — i.e., reuse existing BST nodes by modifying their pointers.

This transformation mimics what happens in databases or memory compaction systems when tree-based indexes are flattened for sequential access.

Data Model

Each BST node will store:

```
int key;  
std::string value;  
BSTNode *left;  
BSTNode *right;
```

Implement the usual BST functionalities:

- `insert(int key, std::string value)`

- `inorderTraversal()` → to display sorted order
- `displayStructure()` → (optional) to visualize the tree

These serve as baseline verification before conversion.

Convert the BST into a sorted linked list **in-place**.

- The `right` pointer of each node acts as the “next” pointer in the list.
- The `left` pointer can be set to `nullptr` (or used as “prev” if doubly linked).
- The function should return a pointer to the **head of the list** (smallest key).
- The list must be sorted in ascending order of keys.

Test Cases

```
TEST(BSTToLinkedListTest, BasicInsertionAndTraversal) {
    BST tree;

    int keys[] = {40, 20, 60, 10, 30, 50, 70};

    int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)
        tree.insert(keys[i], "val");

    std::ostringstream out;

    inorderTraversal(tree.root, out);

    std::string expected = "10 20 30 40 50 60 70 ";

    EXPECT_EQ(out.str(), expected);
}
```

```
}
```

```
TEST(BSTToLinkedListTest, ConvertBalancedTreeToLinkedList) {

    BST tree;

    int keys[] = {40, 20, 60, 10, 30, 50, 70};

    int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        tree.insert(keys[i], "val");

    BSTNode* head = tree.convertToSortedLinkedList();

    EXPECT_TRUE(isSortedLinkedList(head));

    std::string result = linkedListToString(head);

    std::string expected = "10 -> 20 -> 30 -> 40 -> 50 -> 60 -> 70 ->
NULL";

    EXPECT_EQ(result, expected);
}
```

```
TEST(BSTToLinkedListTest, EmptyTreeConversion) {

    BST tree;
```

```
BSTNode* head = tree.convertToSortedLinkedList();

EXPECT_EQ(head, nullptr);

}
```

```
TEST(BSTToLinkedListTest, SingleNodeConversion) {

    BST tree;

    tree.insert(10, "only");

    BSTNode* head = tree.convertToSortedLinkedList();

    ASSERT_NE(head, nullptr);

    EXPECT_EQ(head->key, 10);

    EXPECT_EQ(head->right, nullptr);

    std::string result = linkedListToString(head);

    EXPECT_EQ(result, "10 -> NULL");

}
```

```
TEST(BSTToLinkedListTest, LeftSkewedTreeConversion) {

    BST tree;

    int keys[] = {50, 40, 30, 20, 10};

    int n = sizeof(keys)/sizeof(keys[0]);
```



```

    for (int i = 0; i < n; i++)

        tree.insert(keys[i], "val");

    BSTNode* head = tree.convertToSortedLinkedList();

    EXPECT_TRUE(isSortedLinkedList(head));

    std::string result = linkedListToString(head);

    std::string expected = "10 -> 20 -> 30 -> 40 -> 50 -> NULL";

    EXPECT_EQ(result, expected);
}

```

```

TEST(BSTToLinkedListTest, RightSkewedTreeConversion) {

    BST tree;

    int keys[] = {10, 20, 30, 40, 50};

    int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        tree.insert(keys[i], "val");

    BSTNode* head = tree.convertToSortedLinkedList();

    EXPECT_TRUE(isSortedLinkedList(head));
}

```

```

    std::string result = linkedListToString(head);

    std::string expected = "10 -> 20 -> 30 -> 40 -> 50 -> NULL";

    EXPECT_EQ(result, expected);
}

```

```

TEST(BSTToLinkedListTest, RandomTreeConversion) {

    BST tree;

    int keys[] = {25, 15, 50, 10, 22, 35, 70, 4, 12, 18, 24, 31, 44, 66,
90};

    int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        tree.insert(keys[i], "val");

    BSTNode* head = tree.convertToSortedLinkedList();

    EXPECT_TRUE(isSortedLinkedList(head));

    // Head should point to smallest element (4)

    ASSERT_NE(head, nullptr);

    EXPECT_EQ(head->key, 4);
}

```

```

TEST(BSTToLinkedListTest, DuplicatesHandledProperly) {

    BST tree;

    int keys[] = {40, 20, 60, 20, 60, 40}; // duplicates

    int n = sizeof(keys)/sizeof(keys[0]);

    for (int i = 0; i < n; i++)

        tree.insert(keys[i], "val");

    BSTNode* head = tree.convertToSortedLinkedList();

    EXPECT_TRUE(isSortedLinkedList(head));

    // Expect no cycles or unsorted nodes

    int prev = -9999;

    BSTNode* curr = head;

    while (curr) {

        EXPECT_GE(curr->key, prev);

        prev = curr->key;

        curr = curr->right;

    }

}

```

```

TEST(BSTToLinkedListTest, StructuralIntegrityCheck) {

```

```
BST tree;

int keys[] = {40, 10, 60, 5, 20, 50, 70};

int n = sizeof(keys)/sizeof(keys[0]);

for (int i = 0; i < n; i++)

    tree.insert(keys[i], "val");


// Count nodes before conversion

std::ostringstream out;

inorderTraversal(tree.root, out);

int nodeCountBefore = std::count(out.str().begin(), out.str().end(), '
') - 1;


BSTNode* head = tree.convertToSortedLinkedList();


// Count nodes after conversion

int count = 0;

while (head) {

    count++;

    head = head->right;

}


EXPECT_EQ(count, nodeCountBefore);

}
```

